Smart Driver Communications Interface Protocol

Phase 1 Abridged Edition

1. Overview

This document defines a peer-to-peer communications protocol that is used for communications between Communications Interface Module (CIM) devices and Driver Control Module (DCM) devices in a smart driver application.

The full-duplex serial communications link between devices will be point-to-point and consist of exactly two link partners.

This protocol is message based and uses a request – response communications method for all normal message exchanges. Special asynchronous notification messages will use a one-way request only method where no response is sent back.

A request message is sent by a requestor (typically a CIM) to a receiver (typically a DCM). Upon receipt of the request message the receiver will process the request and send back an appropriate response message, or an error message in the case of an error occurring while processing the request message.

Since this protocol is peer-to-peer there is no notion of a master or slave, any device may act as a requestor and initiate a message exchange. However, in typical operation CIM devices will be the requestor in most cases. The expected use case of a DCM device acting as the requestor is when sending an asynchronous notification message to a CIM.

2. Definitions

CIM: Communications Interface Module. A device which implements an external communications interface, for instance DMX or BLE. The CIM may be an external dongle, an internal daughtercard or soldered directly to the PCB in a driver. A CIM contains a microcontroller and any other specialized circuitry it requires to implement the external communications interface it was designed to handle.

DCM: Driver Control Module. A device which controls one or more LEDs, typically a microcontroller and it's supporting circuitry within a driver. It is responsible for controlling LED intensity, color and/or other parameters as well as reporting back status information.

Requestor: A device that is making a request to another device.

Receiver: A device that is receiving a request from another device.

Channel: A single unit of control for an LED chain. Provides on/off control and current regulation for a single LED chain.

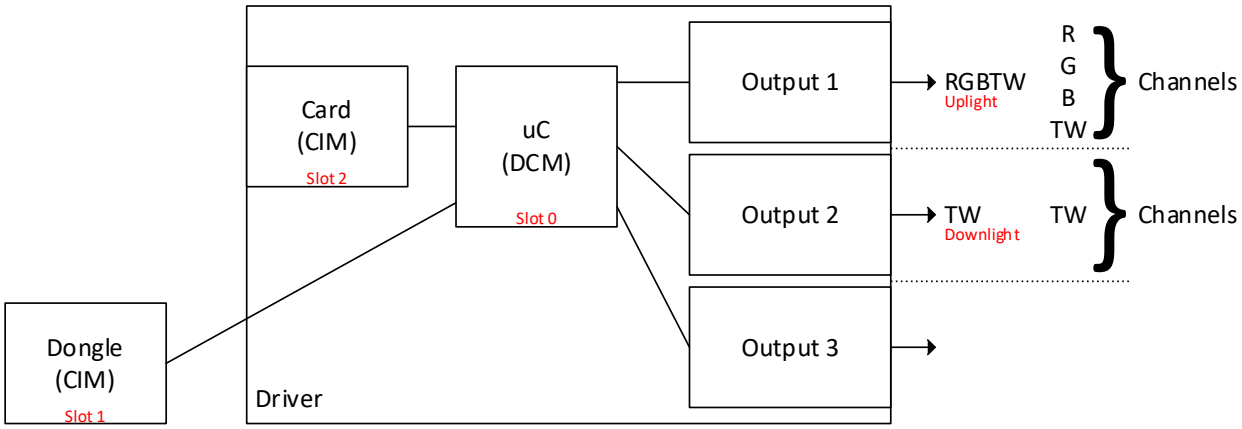
Output: A collection of channels implementing higher level control functions. For instance, an RGBTW output consists of 5 channels that are controlled together to provide the desired light intensity and color.

Slot: An addressing scheme for identifying DCM and CIM devices. The DCM will always be slot 0.

Driver: A device that provides the power regulation and control functions of a lighting fixture. A driver may contain multiple outputs each consisting of multiple drive channels. A driver contains one DCM and may support multiple simultaneous CIM interfaces.

2.1. Logical Diagram

This diagram shows a graphical illustration of the above definitions.



2.2. Numeric Values

Where numeric values are used in this document their base will be indicated as follows

Base	Example	Notes
Decimal	1	Just the number, no prefix
Hex	0x01	Hex values will be prefixed with a 0x
Binary	0b0001	Binary values will be prefixed with a 0b

2.3. Data Types

Definitions for common data types used in this document

Type	Description
U8	Unsigned 8-bit value
U16	Unsigned 16-bit value
U32	Unsigned 32-bit value
U8[]	Array of U8 values
Bit-Field	Bit-field
String	Array of characters, 7-bit ASCII encoding, see notes below
Bool	Boolean value, U8, 0=False, 1-255=True

2.3.1. Byte Ordering

The encoding of multi-byte values will use Big-Endian byte ordering.

2.3.2. ASCII:

https://en.wikipedia.org/wiki/ASCII

All characters within a string must be limited to the base 7-bit ASCII characters, 0x00-0x7F.

Strings must be null terminated.

All unused bytes in a string buffer must be filled with 0x00.

3. Communications Interface

3.1. Physical interface

This protocol uses a Serial UART based interface with the following properties

Baud rate: 115200

Data Bits: 8

Parity: None

Stop Bits: 1

Hardware Handshake: None

Signals: TX & RX only no flow control signals required

3.2. Timeouts and Retries

Response Timeout: 1 second

Retry Count: 2

A Requestor should use the above specified default Timeout value while waiting for a response from a Receiver.

A Requestor should implement retry logic that will re-send a Request Message a specified number of additional times after:

- A timeout occurs.
- Receiving an Error Response Message and the Error Code indicates that a retry would be worth the effort. This will be Command Code and context specific.

Individual Command Codes may specify different Timeout and Retry values if needed.

3.3. Communications Flow – Request → Response

A Requestor device will send a Request Message to the Receiver device and wait for a response from the receiver.

A Receiver device will listen for Request Messages and upon receiving one process the request and send back to the Requestor a response.

- If the request was successfully processed a Response Message will be sent with the required information.
- If an error occurs while processing the request an Error Response Message with the Error Code set to an appropriate value will be sent.

The Requestor device, upon receiving a response, will handle the response as necessary.

- In the case of an Error or Timeout the requestor may elect to retry to send the request again repeating this communication flow.

4. Communications Packet Structure

Index	Field	Bytes	Type	Notes
1	Start Code	1	U8	Fixed value (0xF1)
2	Protocol Version	1	U8	Fixed value (1)
3	Flags	2	Bit-Field	See definition below
4	Transaction ID	1	U8	See definition below
5	Command Code	2	U16	See definition below
6	Payload Length	2	U16	Length of the payload
7	Payload	0-538	U8[]	Optional
8	Checksum	2	U16	Computed on Indexes 2 through 7 inclusive
9	Stop Code	1	U8	Fixed value (0xF2)

Packet Length: 12 + Payload Length

Max Payload Size: 538 Bytes (512 + 26)

Max Packet Size: 550 Bytes

Note: The use of unique Start and Stop Codes make the packet "self-starting" in the sense that the beginning and end of the packet are determined by looking for unique bytes that will only appear at the beginning and end of the packet, not within the packet.

4.1. Start Code - [1 Byte / U8]

Fixed value: 0xF1

Reserved value used to denote the beginning of a communications packet. This reserved value may only appear in a packet as the start code. If this reserved value must appear anywhere else in the packet it must be escaped. Refer to section "8. Escaping Reserved Bytes" for details.

4.2. Protocol Version - [1 Byte / U8]

Fixed value: 1

Indicates the structural formatting and expected interpretation of this communications packet.

This value will be increased if a future breaking change is made to this packet structure or the meaning of any of the fields change.

This value must be checked when receiving a packet and if the packet is encoded using a version that is not supported by the receiver, the receiver must discard the packet and not attempt to interpret its contents.

A device may support multiple versions of the protocol for backward compatibility and interoperability.

4.3. Flags - [2 Bytes / Bits]

Bit-field containing various parameters that are used to interpret the packet.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID	Reserved										Slot			Type		

This value must be checked when receiving a packet and if an invalid value is found in any field, the receiver must discard the packet and not attempt to interpret its contents.

4.3.1. Reserved Bits - [10 Bits]

All reserved bits must be set to a value of 0.

4.3.2. Slot - [3 Bits]

Fixed value: 0 (Driver)

Will be used in a future revision of this protocol document for multi-slot addressing.

4.3.3. Type - [3 Bits]

Defines what type of packet this is.

Value	Type	Notes
0	Request	Denotes a packet containing a request from a requestor to a receiver
1	Response	Denotes a packet containing a response to a request from a receiver back to the requestor
2	Error	Denotes a packet containing an error response to a request from a receiver back to the requestor
3 - 7	Reserved	Reserved for future use

4.4. Transaction ID - [1 Byte / U8]

An incrementing value used to match requests and responses (0-255, wraps around). The value is set by the requestor when sending a request packet, the receiver when responding to the request must return the same transaction ID value that was in the original request packet. This also applies to error responses.

4.5. Command Code - [2 Bytes / U16]

A command code is a unique value that identifies a requested action that a receiver must process and respond to. Refer to section "9. Command Code Definition" for details.

4.6. Payload Length - [2 Bytes / U16]

Length of the Payload in bytes (0 to 538). The payload length is defined on a per command & response basis. Refer to section "9. Command Code Definition" for details.

Note: This is not the overall packet size, it just specifies the payload size.

4.7. Payload - [0-538 Bytes / U8[]]

The payload contents are defined on a per command & response basis. Refer to section "9. Command Code Definition" for details.

4.8. Checksum - [2 Bytes / U16]

A 16-bit CRC is calculated over all previous bytes in the packet, starting from the Protocol Version field (index 2) up to and including the last byte of the Payload (index 7).

Refer to section "10. CRC Implementation Definition" for details.

4.9. Stop Code - [1 Byte / U8]

Fixed value: 0xF2

Reserved value used to denote the end of a communications packet. This reserved value may only appear in a packet as the stop code. If this reserved value must appear anywhere

else in the packet it must be escaped. Refer to section "8. Escaping Reserved Bytes" for details.

4.10. Packet Structural Validation

The steps below outline the procedure to be used to verify the structural integrity of any received communications packet. The steps must be performed in the listed order.

Note: The received packet buffer must not contain the Start and Stop codes and all data must be un-escaped before proceeding with the verification steps listed below.

Note: In general, if the packet structure is not correct the packet should be discarded. The reasoning being that the packet was most likely damaged during transmission and attempting to interpret potentially bad information may lead to undesirable consequences.

Steps to verify a received packet:

1. Check if the total received packet length is at least 10 bytes
    - If not, discard the packet
2. Check if the Protocol Version is 1
    - If not, discard the packet
3. Check if Flags.Reserved and Flags.Slot are 0 and Flags.Type is 0, 1 or 2
    - If not, discard the packet
4. Check if the actual Payload length matches the specified Payload Length
    - If not, discard the packet
5. Compute the checksum of all received packet data bytes except for the last two bytes, these are the expected Checksum value
    - Verify the computed checksum value against the expected Checksum value
        - If they don't match, discard the packet

After the packet structure has been validated it can be sent on to higher level command validation and processing steps. These next steps will validate the Command Code and Payload data. See the respective sections below for details.

4.11. Packet Handling

This section defines the logic of how a serial stream of data bytes received via the USART should be parsed into packets for subsequent processing.

- Any data bytes received between a Start Code and a Stop Code are considered to be “inside” the packet. All other received data bytes are considered to be “outside” the packet.
- When a Start Code is received it indicates the beginning of a new packet (we are now inside the packet).
    - If we were inside a previous packet when receiving a new Start Code, the previous packet must be discarded, the packet buffer flushed and a new packet started.
- When a Stop Code is received while inside a packet it indicates the end of the packet, and any packet data received thus far must now be processed.
    - Upon receiving the Stop Code, the packet must be immediately processed, no additional delays or timeout should be used.
    - After processing the received packet, the packet buffer must be flushed and made ready for the next packet.
- Any data received while outside of a packet must be discarded and not saved into the packet buffer.
    - While outside of a packet the Start Code is the only byte value that is being searched for, all other byte values must be discarded including the reserved values 0xF0(Reserved), 0xF2(Stop Code) and 0xF3(Escape Code).
- Any data received while inside of a packet must be un-escaped and stored into the packet buffer.
- Start and Stop Code values must not be placed into the packet buffer.
    - They are only used to frame the packet and are not needed beyond this point.

5. Request Message

A Request Message is used to send a request to a receiving device. The Request Message uses the standard Communications Packet Structure as defined above as its base. Refer to section "4. Communications Packet Structure" for details.

5.1. Include the following information in the Request Message

- Set Protocol Version to the current version of the protocol as defined above.
- Add the Flags value
  - Set Slot = 0
  - Set Type = 0 (Request)
- Set the Transaction ID to the next value in sequence
- Set the Command Code to the value of the desired command
- Set the Payload Length based on the specified Command Code
- Set the Payload based on the specified Command Code
- Compute and add the Checksum

5.2. Validation of a received Request Message

When receiving a Request Message, the receiver must validate the following items:

Verify the received packet per the steps outline in section "4.10. Packet Structural Validation".

Verify the Command Code is supported by the receiver, if not then return an Error Response Message with the Error Code set to ERR_UNSUPPORTED_COMMAND.

Verify that the Payload Length is what is expected for the Command Code, if not then return an Error Response Message with the Error Code set to ERR_PAYLOAD_LENGTH. Refer to section "9. Command Code Definition" for details.

6. Response Message

A Response Message is used to send the response to a request message back to the requesting device. The Response Message uses the standard Communications Packet Structure as defined above as its base. Refer to section "4. Communications Packet Structure" for details.

If a Response Message is received from a peer without having sent a Request Message, the Response Message should be discarded and no further action taken.

6.1. Include the following information in the Response Message

- Set Protocol Version to the current version of the protocol as defined above.
    - Or to the Protocol Version from the Request Message you are responding to if you support that requested Protocol Version. Note in that case the entire Response Message must be formatted according to that Protocol Version, refer to the appropriate version of this document for that Protocol Version.
- Add the Flags value
    - Set Slot = 0
    - Set Type = 1 (Response)
- Set the Transaction ID to the value from the Request Packet you are responding to
- Set the Command Code to the value from the Request Packet you are responding to
- Set the Payload Length based on the defined response of the requested Command Code
- Set the Payload based on the defined response of the requested Command Code
- Compute and add the Checksum

6.2. Validation of a received Response Message

When receiving a Response Message, the receiver must validate the following items:

Verify the received packet per the steps outline in section "4.10. Packet Structural Validation".

Verify that the Command Code matches the one in the Request Message to which this is the response.

Verify that the Transaction ID matches the one in the Request Message to which this is the response.

Verify that the Payload Length is what is expected for a response to the Command Code.

If any of the above verifications fail, then discard the message and take appropriate action, which may be:

- Simply discard the response and carry on.
    - May additionally log the error or notify someone.
- Attempting to retry sending the Request Message to which this was the response.
- Some commands may define actions to take in such events.

7. Error Response Message

An Error Response Message is used to send a response back to the requesting device indicating that an error has occurred while trying to process the Request Message. The Error Response Message uses the standard Communications Packet Structure as defined above as its base. Refer to section "4. Communications Packet Structure" for details.

If an Error Response Message is received from a peer without having sent a Request Message, the Error Response Message should be discarded and no further action taken.

7.1. Include the following information in the Error Response Message

- Set Protocol Version to the current version of the protocol as defined above.
    - Or to the Protocol Version from the Request Message you are responding to if you support that requested Protocol Version. Note in that case the entire Error Response Message must be formatted according to that Protocol Version, refer to the appropriate version of this document for that Protocol Version.
- Add the Flags value
    - Set Slot = 0
    - Set Type = 2 (Error)
- Set the Transaction ID to the value from the Request Packet you are responding to
- Set the Command Code to the value from the Request Packet you are responding to
- Set the Payload Length as defined below
- Set the Payload as defined below
- Compute and add the Checksum

7.2. Error Response Message Payload

Index	Field	Bytes	Type	Notes
1	Error Code	4	U32	
2	Description Length	2	U16	
3	Error Description	0-512	String	Optional

Error Response Message Payload Length: 5 + Size of the Error Description

7.2.1. Error Code - [4 Bytes / U32]

A unique value that identifies a standard defined error type. Refer to section "11. Standard Error Codes" for a list of defined Error Codes.

7.2.2. Description Length - [2 Byte / U16]

Length of the Error Description in bytes (0 to 512).

7.2.3. Error Description - [0-512 Bytes / String]

Optional descriptive text that may provide additional context about the error that has occurred.

7.3. Validation of a received Error Response Message

When receiving an Error Response Message, the receiver must validate the following items:

Verify the received packet per the steps outline in section "4.10. Packet Structural Validation".

Verify that the Command Code matches the one in the Request Message to which this is the response.

Verify that the Transaction ID matches the one in the Request Message to which this is the response.

Verify that the Payload Length is what is expected for an Error Response Message, see above.

If any of the above verifications fail, then discard the message and take appropriate action, which may be:

- Simply discard the response and carry on.
    - May additionally log the error or notify someone.
- Attempting to retry sending the Request Message to which this was the response.
- Some commands may define actions to take in such events.

7.4. Handling of a received Error Response Message

When receiving an Error Response Message in response to a Request Message the requester must take appropriate action based on the returned Error Code, which may be:

- Simply discard the response and carry on.
    - May additionally log the error or notify someone.
- Attempting to retry sending the Request Message to which this was the response.
    - Perhaps after adjusting the Request Message based on the Error Code
- Some commands may define actions to take in response to certain Error Codes.

8. Escaping Reserved Bytes

8.1. Reserved Bytes

The following reserved byte values may not appear in the packet except as defined below

Value	Notes
0xF0	Reserved for future use
0xF1	Used as the packet Start Code
0xF2	Used as the packet Stop Code
0xF3	Used as the Escape Code

8.2. Byte Escaping

If any of the reserved byte values (see list above) needs to appear within the packet at indexes 2 through 7, an escaping technique must be employed to transform the reserved byte value into a two-byte sequence. Byte escaping must be performed on the entire packet, except for the Start and Stop Code indexes, including the checksum after it is calculated.

To escape a value, expand it into a two-byte sequence containing the Escape Code (0xF3) and the original value AND'd with 0x0F.

To un-escape the two-byte sequence, remove the Escape Code (0xF3) and OR the second value with 0xF0 to get the original byte value back.

For example, if the reserved value 0xF1 (Start Code), needs to appear somewhere in the packet, other than as the Start Code, it would need to be escaped into the two-byte sequence 0xF3 0x01 when transmitted. To un-escape the value, the receiver would discard the Escape Code (0xF3) and OR the second value (0x01) with 0xF0 to get the original value of 0xF1 back.

The transmission software should, as each byte is placed into the transmission buffer, examine the byte and determine if it needs to be escaped, excluding the actual packet Start and Stop Codes in their designated indexes. The byte escaping can be undone by the receiving software before the bytes are stored in the receive buffer.

8.3. Handling of unused Reserved Bytes

When the device receives a reserved byte code that is defined as "Reserved for future use" it should be discarded no matter when it is received, inside or outside of a packet.

8.4. Byte Escaping Examples

A value of 0xF1 appears in the middle of the packet

```
Original: 0xF1 0x00 0x10 0xF1 0x00 0x01 0x01 0x00 0xAA 0x94 0xA8 0xF2
 Escaped: 0xF1 0x00 0x10 0xF3 0x01 0x00 0x01 0x01 0x00 0xAA 0x94 0xA8 0xF2
```

A value of 0xF3 appears in the middle of the packet

```
Original: 0xF1 0x00 0x10 0xBE 0x00 0x01 0x02 0x00 0xAA 0xF3 0x57 0x6A 0xF2
 Escaped: 0xF1 0x00 0x10 0xBE 0x00 0x01 0x02 0x00 0xAA 0xF3 0x03 0x57 0x6A 0xF2
```

A value of 0xF2 appears in the middle of the packet

```
Original: 0xF1 0x00 0x10 0xBE 0x00 0x01 0x01 0x00 0xAA 0x57 0xF2 0xF2
 Escaped: 0xF1 0x00 0x10 0xBE 0x00 0x01 0x01 0x00 0xAA 0x57 0xF3 0x02 0xF2
```

9. Command Code Definition

A command code is a unique 16-bit value that identifies a requested action that a receiver must process and respond to.

9.1. Command Table

A listing all available command code that have been defined. See below for information on adding new commands.

Command Code	Group	Method	Command Name
Operation			
0x0000	0x00	0	???
Discovery			
0x0800	0x08	0	Identify
0x0801	0x08	1	Get Features List
0x0802	0x08	2	Get Feature Detail
0x0803	0x08	3	Get Command List
Properties/Profiles			
0x1000	0x10	0	Get Properties List
0x1001	0x10	1	Get Property
0x1002	0x10	2	Set Property
0x1003	0x10	3	Get Active Profile
0x1004	0x10	4	Set Active Profile
OTA			
0x1800	0x18	0	Reboot
0x1801	0x18	1	Begin OTA
0x1802	0x18	2	Send OTA Data
0x1803	0x18	3	Finish OTA
Telemetry			
0x2000	0x20	0	???
Debug/Test			
0x2800	0x28	0	Ping
Async Notification			
0x3000	0x30	0	???

Command Code	Group	Method	Command Name
Temporary Development			
0x38xx	0x38	0-255	Not defined in this document

See below for details on each defined command including their Request and Response Message Payload definitions.

9.2. Command Code Bit-Field Definition

The command code is composed of a set of bit fields that allow for organizing related commands into groups to ease management and facilitate adding new commands. Use of the defined command codes does not require knowledge of these bit-fields, they are for creating and managing command codes.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID	Group								Method							

9.2.1. Group - [8 Bits]

Defines sets of related methods. Below is a list of the defined groups.

Value	Group	Notes
0x00	Operation	Methods that control normal operation: intensity, color temperature, etc.
0x08	Discovery	Device discovery and feature flag management
0x10	Properties/Profiles	Device configuration management
0x18	OTA	OTA firmware updates
0x20	Telemetry	Sensor management: LED life, temperatures, etc.
0x28	Debug/Test	Test and debug functions
0x30	Async Notification	Messages used for asynchronous notifications. Methods defined in the group are one-way request only messages that have no response.
0x38	Temporary Development	Methods that are used only for temporary development and testing and should not make it into the final release. Methods defined under this group do not require registration in this document as they are considered temporary.
0x40 – 0xFF	Reserved	Reserved for future use

9.2.2. Method - [8 Bits]

Identifies the requested action that a receiver must process and respond to.

There is a set of methods defined within each group. The method value must be unique within a group.

9.3. Identify (0x)

Insert description here

9.4. Get Features List (0x)

Insert description here

9.5. Get Feature Detail (0x)

Insert description here

9.6. Get Command List (0x)

Insert description here

9.7. Get Properties List (0x)

Insert description here

9.8. Get Property (0x)

Insert description here

9.9. Set Property (0x)

Insert description here

9.10. Ping (0x2800)

This command can be used to test communications with a peer.

Any Test Data received in the Request Message will be echoed back to the requestor in the Response Message.

If all required verification steps have passed, then send a Response Message.

9.10.1. Request Message Payload

Index	Field	Bytes	Type	Notes
1	Test Data	0-538	U8	

9.10.1.1. Test Data - [0-538 Bytes / U8]

Optional data to be echoed back to the requestor.

9.10.2. Response Message Payload

Index	Field	Bytes	Type	Notes
1	Test Data	0-538	U8	

9.10.2.1. Test Data - [0-538 Bytes / U8]

The supplied Test Data from the Request Message.

9.10.3. Example

Send a Ping Request Message with the following parameters (Transaction ID=0x84, Test Data=1,2,3) and receive a success response.

Request Message: (Example hex data shown not escaped)

```
0xF1 0x01 0x00 0x00 0x84 0x28 0x00 0x00 0x03 0x01 0x02 0x03 0xF9 0x1C 0xF2
```

Response Message:

```
0xF1 0x01 0x00 0x01 0x84 0x28 0x00 0x00 0x03 0x01 0x02 0x03 0x12 0x3F 0xF2
```

9.11. Reboot (0x1800)

Perform a reboot of the device after a specified delay.

Upon receipt of this message, abort any currently active reboot countdowns.

If the Delay value is greater than 0 then after sending a Response Message the device must wait for the specified Delay number of seconds, then reboot.

If all required verification steps have passed, then send a Response Message.

9.11.1. Request Message Payload

Index	Field	Bytes	Type	Notes
1	Delay	1	U8	

9.11.1.1. Delay - [1 Byte / U8]

Specifies the amount of time in seconds to delay before rebooting the device.

A Delay value of zero is used to abort any active reboot countdowns and not start a new one.

9.11.2. Response Message Payload

No Payload.

9.11.3. Example

Send a Reboot Request Message with the following parameters (Transaction ID=0xEA, Delay=5) and receive a success response.

Request Message: (Example hex data shown not escaped)

```
0xF1 0x01 0x00 0x00 0xEA 0x18 0x00 0x00 0x01 0x05 0x9B 0x88 0xF2
```

Response Message:

```
0xF1 0x01 0x00 0x01 0xEA 0x18 0x00 0x00 0x00 0xD0 0x8B 0xF2
```

9.12. Begin OTA (0x1801)

Initiate an OTA firmware update process with the receiver.

Starts a new OTA session. An OTA session is defined as the entire sequence of Begin OTA, Send OTA Data and Finish OTA messages required to transfer the OTA firmware image to the receiver. An OTA session starts with the Begin OTA message and ends with the Finish OTA message. The OTA session will contain various control and state parameters that are used throughout the OTA process.

If an OTA session is already in process, then abort the in-process session and start a new one.

If all required verification steps have passed, then send a Response Message.

9.12.1. Request Message Payload

Index	Field	Bytes	Type	Notes
1	Flags	1	Bit-Field	
2	Target	1	U8	
3	Region	1	U8	
4	Firmware MD5	16	U8[]	From firmware image header
5	Firmware Length	4	U32	From firmware image header
6	Firmware ID	4	U32	From firmware image header
7	Version Major	2	U16	From firmware image header
8	Version Minor	2	U16	From firmware image header
9	Version Patch	2	U16	From firmware image header
10	Version Build	2	U16	From firmware image header
11	Version Label	16	String	From firmware image header

TODO: Version info likely to change

9.12.1.1. Flags - [1 Byte / Bit-Field]

Bit-field containing flags used to control operation of the firmware update process.

Bit	7	6	5	4	3	2	1	0
ID	Reserved						Downgrade	Force

9.12.1.1.1. Reserved – [6 Bits]

All reserved bits must be set to a value of 0.

9.12.1.1.2. Downgrade – [1 Bit / Bool]

If this flag is set, ignore the Version information. This allows for the installation of firmware images that are equal to or older than what is currently installed. The Firmware ID must still be verified.

9.12.1.1.3. Force – [1 Bit / Bool]

If this flag is set, ignore the Firmware ID and Version information and install the OTA image anyway.

9.12.1.2. Target - [1 Byte / U8]

Specifies the destination target for the firmware image. In most cases this is just 0 for a device that contains only a single microcontroller.

The Target value must be verified. If it is not valid then an Error Response Message must be returned with the Error Code set to ERR_OTA_UNSUPPORTED_TARGET.

9.12.1.3. Region - [1 Byte / U8]

Specifies the region within the destination target for the firmware image. This is used to specify whether the firmware image is intended for the main application region or the boot loader region.

Value	Region
0	Main Application
1	Bootloader
2-255	Other target specific memory region

The Region value must be verified. If it is not valid then an Error Response Message must be returned with the Error Code set to ERR_OTA_UNSUPPORTED_REGION.

9.12.1.4. Firmware MD5 - [16 Bytes / U8[]]

Expected MD5 hash value of the firmware image. The hash does not include the image header, it is only computed on the image contents, with the length as specified by the Length field.

This value must be held on to as it will be needed in the future with the Finish OTA command.

9.12.1.5. Firmware Length - [4 Bytes / U32]

The length of the firmware image. The length does not include the image header, just the image contents.

This value must be held on to as it will be needed in the future with the Send OTA Data and Finish OTA commands.

The specified image Firmware Length must be verified to ensure that it will fit into the specified Target and Region. If the specified image Length is too large, then an Error Response Message must be returned with the Error Code set to ERR_OTA_IMAGE_OVERSIZE.

9.12.1.6. Firmware ID - [4 Bytes / U32]

A unique value that is used to determine compatibility of a firmware image for the specified Target and Region. This is used to prevent the wrong image from being loaded into a device.

The Firmware ID must be verified to ensure that it is compatible with the specified Target and Region. If it is not compatible, then an Error Response Message must be returned with the Error Code set to ERR_OTA_INCOMPATIBLE_ID.

The Firmware ID verification must be skipped if the Force bit is set in the Flags field, see above for details.

Refer to section "12. Registered Firmware IDs" for a list of registered Firmware IDs.

9.12.1.7. Version Major - [2 Bytes / U16]

TODO: Finalize versioning

The Major component of the Full Firmware Version.

The Full Firmware Version is defined as:

"Version Major . Version Minor . Version Patch . Version Build - Version Label"

Refer to section "15. Firmware Versioning" for details on firmware version definition, verification and comparison logic.

By default, only newer firmware images may be installed. A firmware image with a version that is equal to or older than the current installed version may not be installed.

The Full Firmware Version of the OTA image must be compared against the Full Firmware Version of the current running firmware in the specified Target and Region. If the Full Firmware Version of the OTA image is newer, then proceed with the installation of the OTA image otherwise return an Error Response Message with the Error Code set to ERR_OTA_OLD_VERSION.

The Full Firmware Version verification must be skipped if either the Force or Downgrade bits are set in the Flags field, see above for details.

9.12.1.8. Version Minor - [2 Bytes / U16]

The Minor component of the Full Firmware Version.

Refer to the Version Major section above for the Full Firmware Version definition and verification logic.

9.12.1.9. Version Patch - [2 Bytes / U16]

The Patch component of the Full Firmware Version.

Refer to the Version Major section above for the Full Firmware Version definition and verification logic.

9.12.1.10. Version Build - [2 Bytes / U16]

The Build component of the Full Firmware Version.

Refer to the Version Major section above for the Full Firmware Version definition and verification logic.

9.12.1.11. Version Label - [16 Bytes / String]

The Label component of the Full Firmware Version.

Refer to the Version Major section above for the Full Firmware Version definition and verification logic.

9.12.2.Response Message Payload

No Payload.

9.12.3.Example

Send a Begin OTA Request Message with the following parameters (Transaction ID=1, Target=1, Region=2, Firmware MD5=8DDD8BE4B179A529AFA5F2FFAE4B9858, Firmware Length=13, Firmware ID=0xAABBCCDD, Version Major=1, Version Minor=2, Version Patch=3, Version Build=4, Version Label=”RC1”) and receive a success response.

Request Message: (Example hex data shown not escaped)

```
0xF1 0x01 0x00 0x00 0x01 0x18 0x01 0x00 0x33 0x00 0x01 0x02 0x8D 0xDD 0x8B 0xE4
0xB1 0x79 0xA5 0x29 0xAF 0xA5 0xF2 0xFF 0xAE 0x4B 0x98 0x58 0x00 0x00 0x00 0x0D
0xAA 0xBB 0xCC 0xDD 0x00 0x01 0x00 0x02 0x00 0x03 0x00 0x04 0x52 0x43 0x31 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x1E 0xC0 0xF2
```

Response Message:

```
0xF1 0x01 0x00 0x01 0x01 0x18 0x01 0x00 0x00 0x30 0x48 0xF2
```

9.13. Send OTA Data (0x1802)

Send chunks of OTA image data to the receiver.

If an OTA session is not in process, then an Error Response Message must be returned with the Error Code set to ERR_OTA_NOT_ACTIVE.

A Total Bytes Received count must be kept across the entire OTA session. The value is reset on the receipt of a Begin OTA message and is updated for each valid Send OTA Data message. Total Bytes Received += (Payload Length – 4)

The OTA firmware binary Image Data chunks must be sent in order across subsequent Send OTA Data messages. Gaps in the data or overlapping data is not allowed.

If all required verification steps have passed, then send a Response Message and update the Total Bytes Received count.

9.13.1. Request Message Payload

Index	Field	Bytes	Type	Notes
1	Offset	4	U32	
2	Image Data	1-512	U8[]	

9.13.1.1. Offset - [4 Bytes / U32]

The offset in bytes form the beginning of the firmware image binary blob. Used to verify we have not missed or duplicated any data along the way. This does not correspond to a memory load address on the Target; it is just an offset into the binary data.

The Offset value must be compared to the Total Bytes Received count upon the receipt of each new Send OTA Data Message before the count is updated.

- If the Offset value is greater than the Total Bytes Received value, then an Error Response Message must be returned with the Error Code set to ERR_OTA_MISSING_DATA.
- If the Offset value is less than the Total Bytes Received value, then an Error Response Message must be returned with the Error Code set to ERR_OTA_DUPLICATE_DATA.

Verify that Total Bytes Received + (Payload Length – 4) is less than or equal to the Firmware Length value specified in the Begin OTA message. If not, then an Error Response Message must be returned with the Error Code set to ERR_OTA_IMAGE_OVERSIZE.

9.13.1.2. Image Data - [1-512 Bytes / U8[]]

Contains a chunk of the firmware binary image.

The size of the Image Data chunk may be of any value in the range of 1 to 512 bytes; max sized chunks are not required. If the size is outside the allowed range, then an Error Response Message must be returned with the Error Code set to ERR_OTA_CHUNK_SIZE. Image Data chunk size = Payload Length – 4.

The Image Data must be stored in an appropriate location for later use to update the device firmware at the end of the OTA process. If the Image Data chunk cannot be stored for any reason, then an Error Response Message must be returned with the Error Code set to ERR_OTA_STORE_FAIL.

9.13.2. Response Message Payload

No Payload.

9.13.3. Example

Send a Send OTA Request Message with the following parameters (Transaction ID=0xAA, Offset=0x1234, Image Data=0x48,0x65,0x6C,0x6C,0x6F,0x20,0x57,0x6F,0x72,0x6C,0x64, 0x21,0x0A) and receive a success response.

Request Message: (Example hex data shown not escaped)

```
0xF1 0x01 0x00 0x00 0xAA 0x18 0x02 0x00 0x11 0x00 0x00 0x12 0x34 0x48 0x65 0x6C
0x6C 0x6F 0x20 0x57 0x6F 0x72 0x6C 0x64 0x21 0x0A 0xA8 0x45 0xF2
```

Response Message:

```
0xF1 0x01 0x00 0x01 0xAA 0x18 0x02 0x00 0x00 0xAF 0x83 0xF2
```

9.14. Finish OTA (0x1803)

Verify and install the uploaded OTA firmware image

If an OTA session is not in process, then an Error Response Message must be returned with the Error Code set to ERR_OTA_NOT_ACTIVE.

Upon receipt of this request, perform the following verification steps:

- Verify that Total Bytes Received is equal to the Firmware Length value specified in the Begin OTA message. If not, then an Error Response Message must be returned with the Error Code set to ERR_OTA_VERIFY_SIZE.
- Compute an MD5 hash of the received OTA image (based on the above verified length) and verify it is equal to the Firmware MD5 value specified in the Begin OTA message. If not, then an Error Response Message must be returned with the Error Code set to ERR_OTA_VERIFY_HASH.

If all required verification steps have passed, then send a Response Message and:

- End the current OTA session.
- If Install Now is True, then wait for 1 second to give the Response Message time to be fully transmitted then reboot to perform the installation of the new OTA firmware image.
    - If a Begin OTA Request message is received while waiting, abort the wait and don't reboot. Instead start a new OTA session.

9.14.1. Request Message Payload

Index	Field	Bytes	Type	Notes
1	Install Now	1	Bool	

9.14.1.1. Install Now - [1 Byte / Bool]

Specifies if the receiver should reboot and perform the installation of the OTA firmware image after sending the Response Message back to the Requestor.

True: Install OTA image

False: Don't install OTA image

9.14.2.Response Message Payload

No Payload.

9.14.3.Example

Send a Finish OTA Request Message with the following parameters (Transaction ID=0xE3, Install Now=True) and receive a success response.

Request Message: (Example hex data shown not escaped)

```
0xF1 0x01 0x00 0x00 0xE3 0x18 0x03 0x00 0x01 0x01 0x08 0x32 0xF2
```

Response Message:

```
0xF1 0x01 0x00 0x01 0xE3 0x18 0x03 0x00 0x00 0x21 0xA7 0xF2
```

10. CRC Implementation Definition

Details of the CRC algorithm used in the Communications Packet along with example C code which implements the specified CRC algorithm.

10.1. CRC Algorithm Details

Common Name: CRC-16/CCITT-FALSE

Alias: CRC-16/IBM-3740, CRC-16/AUTOSAR

Width: 16 bit

Polynomial: 0x1021 (x^16+x^12+x^5+1)

Initial value: 0xFFFF

Input reflection: False

Output reflection: False

Output XOR: 0x0000

Check Value: 0x29B1

Residue: 0x0000

Calculator: crccalc.com

10.2. CRC-16/CCITT-FALSE Table Based Implementation Code

```c
#include <stdint.h>

uint16_t const crc_ccitt_false_table[256] = {
    0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7,
    0x8108, 0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF,
    0x1231, 0x0210, 0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6,
    0x9339, 0x8318, 0xB37B, 0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE,
    0x2462, 0x3443, 0x0420, 0x1401, 0x64E6, 0x74C7, 0x44A4, 0x5485,
    0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE, 0xF5CF, 0xC5AC, 0xD58D,
    0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6, 0x5695, 0x46B4,
    0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D, 0xC7BC,
    0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
    0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B,
    0x5AF5, 0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12,
    0xDBFD, 0xCBDC, 0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A,
    0x6CA6, 0x7C87, 0x4CE4, 0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41,
    0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD, 0xAD2A, 0xBD0B, 0x8D68, 0x9D49,
    0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13, 0x2E32, 0x1E51, 0x0E70,
    0xFF9F, 0xEFBE, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A, 0x9F59, 0x8F78,
    0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E, 0xE16F,
    0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
    0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E,
    0x02B1, 0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256,
    0xB5EA, 0xA5CB, 0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D,
    0x34E2, 0x24C3, 0x14A0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
    0xA7DB, 0xB7FA, 0x8799, 0x97B8, 0xE75F, 0xF77E, 0xC71D, 0xD73C,
    0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657, 0x7676, 0x4615, 0x5634,
    0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9, 0xB98A, 0xA9AB,
    0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882, 0x28A3,
    0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABBB, 0xBB9A,
    0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92,
    0xFD2E, 0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9,
    0x7C26, 0x6C07, 0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1,
    0xEF1F, 0xFF3E, 0xCF5D, 0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8,
    0x6E17, 0x7E36, 0x4E55, 0x5E74, 0x2E93, 0x3EB2, 0x0ED1, 0x1EF0
};

uint16_t crc_ccitt_false(uint8_t const *buffer, size_t len)
{
    uint16_t crc = 0xFFFF;

    for (size_t i = 0; i < len; i++)
    {
        crc = (crc << 8) ^ crc_ccitt_false_table[(crc >> 8) ^ buffer[i]];
    }

    return crc;
}
```

11. Standard Error Codes

List of the defined error codes and their description.

Error Code	Error Label	Description
0	ERR_UNKNOWN	Unknown error
1	ERR_PROTOCOL_VERSION	Unsupported Protocol Version
2	ERR_PACKET_CHECKSUM	Checksum verification failed
3	ERR_PACKET_LENGTH	Invalid Packet Length
4	ERR_UNSUPPORTED_COMMAND	Unsupported Command Code
5	ERR_PAYLOAD_LENGTH	Payload is not of the correct length for the specified Command Code
6	ERR_OTA_UNSUPPORTED_TARGET	Unsupported OTA Target
7	ERR_OTA_UNSUPPORTED_REGION	Unsupported OTA Region
8	ERR_OTA_IMAGE_OVERSIZE	The OTA image is too large for Target memory Region
9	ERR_OTA_INCOMPATIBLE_ID	Incompatible Firmware ID
10	ERR_OTA_OLD_VERSION	The OTA firmware is older than what is currently installed
11	ERR_OTA_NOT_ACTIVE	OTA session not active
12	ERR_OTA_MISSING_DATA	Missing data
13	ERR_OTA_DUPLICATE_DATA	Duplicate data received
14	ERR_OTA_CHUNK_SIZE	Invalid data chunk size
15	ERR_OTA_STORE_FAIL	Could not store OTA data chunk
16	ERR_OTA_VERIFY_SIZE	Image size does not match expected size
17	ERR_OTA_VERIFY_HASH	Image hash does not match expected hash
?	ERR_	Insert description here

Error codes are U32 values.

12. Registered Firmware IDs

List of registered Firmware IDs

Firmware ID	Description
0x00000000	Insert driver model and usage here

Firmware IDs are U32 values.

Insert description here

13. Feature Flags

A Feature Flag is a unique 32-bit value that …

Insert a definition of what feature flags are and how they are to be used

14. Properties

The properties database contains the configuration settings used to govern the operation of the driver.

15. Firmware Versioning

Definition for standardized firmware versioning

TODO: add formal definition, links to reference standard, etc.

Definition: v{Major}.{Minor}.{Patch}.{Build}-{Label}

Example: v1.2.3.4-RC1

Comparison:

Major, Minor, Patch & Build are U16 values and may be compared as integers

Label is an ASCII string and may only be compared for difference

15.1. Example Code to Determine if an Update is Needed

```
struct version {
    uint16_t major;
    uint16_t minor;
    uint16_t patch;
    uint16_t build;
    char label[16];
};

// Function to determine if an update is needed.
// If the update version is greater than the current version, an update is needed.
// Returns:
//   true if an update is needed
//   false if no update is needed
bool update_needed(struct version *update, struct version *current) {
    // Compare major versions
    if (update->major != current->major)
        return (update->major > current->major) ? true : false;

    // If major versions are equal, compare minor versions
    if (update->minor != current->minor)
        return (update->minor > current->minor) ? true : false;

    // If minor versions are equal, compare patch versions
    if (update->patch != current->patch)
        return (update->patch > current->patch) ? true : false;

    // If patch versions are equal, compare build versions
    if (update->build != current->build)
        return (update->build > current->build) ? true : false;

    // If build versions are equal, compare labels for equality
    return strncmp(update->label, current->label, sizeof(update->label)) == 0 ? false : true;
}
```

16. Firmware Image Files

Insert definition of firmware images files, including header

17. Change Log

Version	Date	Author	Changes
0.1	2025-11-17.1446	LD	Initial Release
0.2	2026-04-21.1353	LD	Added Ping Command Added clarification on packet handling and validation